COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/31/2026	**DATE SUBMITTED:**	08/05/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

```cpp
#include<iostream>
#include<stack>

int main(){

    std::stack<int> stack1;
    std::cout<<"Testing the stack STL."<<std::endl;

    //isEmpty

    std::cout<<"is the stack empty?"<<stack1.empty()<<std::endl;

    //push

    stack1.push(10);
    std::cout<< "The top of the stack is: "<<stack1.top()<<std::endl;
    stack1.push(9);
    std::cout<< "The top of the stack is: "<<stack1.top()<<std::endl;
    stack1.push(8);
    std::cout<< "The top of the stack is: "<<stack1.top()<<std::endl;
    stack1.push(7);
    std::cout<< "The top of the stack is: "<<stack1.top()<<std::endl;
    stack1.pop();
    std::cout<<"The top of the stack is: "<<stack1.top()<<std::endl;
    std::cout<<"is the stack empty?"<<stack1.empty()<<std::endl;
    std::cout<<"The size of the stack is: "<< stack1.size()<<std::endl;
    stack1.emplace(8);
    std::cout<<"The top of the stack is: "<<stack1.top()<<std::endl;
    stack1.emplace(7);
    std::cout<<"The top of the stack is: "<<stack1.top()<<std::endl;
    std::cout<<"The new size of the stack is: "<<stack1.size()<<std::endl;

    std::stack<int> stack2;
    std::cout<<"Stack 2"<< std::endl;

    stack2.emplace(1);
    std::cout<<"The top of stack 2: "<<stack2.top()<<std::endl;
    stack2.emplace(2);
    std::cout<<"The top of stack 2: "<<stack2.top()<<std::endl;
    stack2.emplace(3);
    std::cout<<"The top of stack 2: "<<stack2.top()<<std::endl;
    stack2.emplace(4);
    std::cout<<"The top of stack 2: "<<stack2.top()<<std::endl;

    std::cout<<"Before swapping: "<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;
    stack1.swap(stack2);

    std::cout<<"\nAfter Swapping."<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;
    std::cout<<"The top of stack 1: "<<stack1.top()<<std::endl;

}
```

Figure 1. Code Snippet of ILO A

```
Testing the stack STL.
is the stack empty?1
The top of the stack is: 10
The top of the stack is: 9
The top of the stack is: 8
The top of the stack is: 7
The top of the stack is: 8
is the stack empty?0
The size of the stack is: 3
The top of the stack is: 8
The top of the stack is: 7
The new size of the stack is:
Stack 2
The top of stack 2: 1
The top of stack 2: 2
The top of stack 2: 3
The top of stack 2: 4
Before swapping:
The top of stack 1: 7
The top of stack 1: 7
The top of stack 1: 7
The top of stack 1: 7

After Swapping.
The top of stack 1: 4
The top of stack 1: 4
The top of stack 1: 4
The top of stack 1: 4
```

Figure 2. Snippet of Output ILOA

From what I've learned and noticed for ILO A is that, just like a normal array, we use the stacks as a way for adding something into an array other than append. Stacks follow a LIFO (Last in, First out) rule wherein what you first input would be the last and the last input would be the first. Following this logic, we used new terms like the stack library, initializing the stack, and the functions it has, like top, push, pop, emplace, isEmpty, and isFull.

The output is like this due to the nature of the stack; each time I push a new integer, it would always come out on top, and when a number is popped, then following that was emplace, where it directly adds it on top of the stack, while push is moving an object that already exists.

```cpp
#include <iostream>
#include <stack>

//global declaration
#define maxCap 10

int stackArr[maxCap];
int top = -1, newData;
//prototype functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void display_Stack();

int main(){
// main driver
    while(true)
    {
        int choice;
        std::cout<<"=========================================================="<<std::endl;
        std::cout<<"Stack Operations: \n"<<std::endl;
        std::cout<<"1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit"<<std::endl;
        std::cout<<"=========================================================="<<std::endl;
        std::cin>> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<<"is stack empty?"<< isEmpty()<<std::endl;
            break;
            case 5: std::cout<<"is the stack full?"<< isFull()<<std::endl;
            break;
            case 6: display_Stack();
            break;
            case 7:
                std::cout<<"Exit"<<std::endl;
                exit(0);
            default: std::cout<<"Invalid Choice."<<std::endl;
            break;
        }
    }
    return 0;
}
```

Figure 3. Code Snippet of ILOBP1

```
50
51  //function definition
52  bool isEmpty(){
53      //how do we verify if the stack is empty?
54      if(top == -1) return true;
55      return false;
56  }
57
58  bool isFull(){
59      //how do we verify if the stack if full?
60      if(top == maxCap -1)return true;
61      return false;
62  }
63
64  void push(){
65      //error checking
66      if(isFull()){
67          std::cout<<"Stack overflow."<<std::endl;
68          return;
69      }
70      //pushing to the stack
71      std::cout<<"Enter a new value: "<<std::endl;
72      std::cin>> newData;
73      //how do we insert the data into the stack>
74      stackArr[++top] = newData;
75  }
76  void pop(){
77      //error checking
78      if(isEmpty){
79          std::cout<<"Stack underflow."<<std::endl;
80          return;
81      }
82
83      //Display the value that we are going to pop:
84      std::cout<<"Popping: "<<stackArr[top]<<std::endl;
85
86      //Decrement the top value from the stack
87      top--;
88  }
89
90  void Top(){
91      //error catching:
92      if(isEmpty()){
93          std::cout<<"The stack is empty."<<std::endl;
94          return;
95      }
96      //check the top value:
97      std::cout<<stackArr[top]<<std::endl;
98  }
99
```

Figure 4. Code Snippet of Functions of ILOBP1

```
100  void display_Stack(){
101      //error catching
102      if(isEmpty()){
103          std::cout<<"The stack is empty. \n"<<std::endl;
104          return;
105      }
106      std::cout<<"Stack: "<< std::endl;
107      for(int i = top; i>= 0; i--){
108          std::cout<<stackArr[i]<<" ";
109      }
110      std::cout<<std::endl;
111  }
```

Figure 5. Code Snippet of display_Stack of ILOBP1

```
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
1
Enter a new value:
10
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
1
Enter a new value:
9
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
1
Enter a new value:
8
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
1
Enter a new value:
7
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
1
Enter a new value:
6
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
2
Stack underflow.
```

Figure 6. Snippet of Output of ILOBP1

```
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
3
6
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
4
is stack empty?0
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
5
is the stack full?0
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
6
Stack:
6 7 8 9 10
==================================================
Stack Operations:

1. PUSH 2. POP 3. TOP 4. isEmpty 5. Is Full 6. Display Stack 7. Exit
==================================================
7
Exit
```

Figure 7. Snippet of Code Output of ILOBP1

For ILOBP1, it was not entirely different from the usual syntax and logic of C++. I think what really made the difference for me here was the logic in the display stack, because I tried looking up how to output an array, since I thought it was quite similar, since each element needs to be printed. But when it was time to test it out, the rule of stacks, which was the LIFO logic, was not implemented at all because instead of using top to start from the last element I pushed, I used 0 instead, which in the logic of arrays would display the first thing I pushed into the array.

HOA4p1 > ILOBP2 > main.cpp > ...

```cpp
#include <iostream>
#include "stackList.h"

int main(){

    std::cout<<"--- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); //Top should be 'e'
    displayAll<char>();
    std::cout<<"\n -----Popping all the elements -----\n";

    while(!isEmpty<char>()){
        std::cout<<"Popped: "<<pop<char>()<<std::endl;
    }

    std::cout<<"\n ----- Testing the underflow ----- \n";
    pop<char>();

    return 0;
}

```

Figure 8. Code Snippet of ILO B.2's main.cpp

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template<typename T>
class Node{
    public:
    T data;     //VALUE THAT WENEED TO STORE
    Node* next; //POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

//push
template<typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    //new node points to the current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

//pop
template<typename T>
T pop(){
    //check if the stack is empty:
    if(head<T> == nullptr){
        std::cout<< "stack overflow."<<std::endl;
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

    //move the head to the next node
    head<T> = head<T>->next;
    //free the removed node
    delete temp;
    //return the removed value
    return tempVal;
}

```

Figure 9. Code Snippet of ILO B.2's stacklist.h

```
51  //isEmpty
52  template<typename T>
53  bool isEmpty(){
54      return head<T> == nullptr;
55  }
56
57  //top
58  template<typename T>
59  void Top(){
60      //error catching:
61      if(isEmpty<T>()){
62          std::cout<<"Stack underflow"<<std::endl;
63          return;
64      }
65      //display the Top node
66      std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
67  }
68
69  //display All
70  template<typename T>
71  void displayAll(){
72      if(head<T> == nullptr){
73          std::cout<<"Stack underflow \n";
74          return;
75      }
76      std::cout<<"Displaying the stack: \n";
77      Node<T>* temp = head<T>;
78      while(temp !=nullptr){
79          std::cout<< temp->data << "\n";
80          temp = temp->next;
81      }
82      std::cout<<"\n";
83  }
84  #endif
85
```

Figure 10. Code Snippet of ILO B.2's stacklist.h

```
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of Stack: e
Displaying the stack:
e
p
c

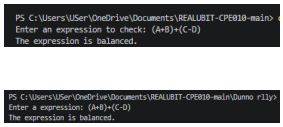
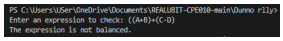
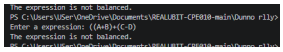
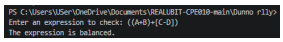
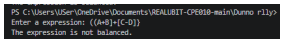
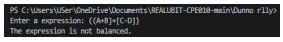

 -----Popping all the elements -----
Popped: e
Popped: p
Popped: c

 ----- Testing the underflow -----
stack underflow.
```

Figure 11. Snippet of Output of ILO B.2

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	 	In case 1, no errors were reached, and it caused "The expression is balanced." This is because both the conditions of whether or not the number of characters is equal to each other and whether or not they pair with each other were proven to be false.
((A+B)+(C-D)	N	 	In case 2, we reach an output of "The expression is not balanced." Primarily because the number of characters does not equal. Therefore, the line **if(!isEmpty()) result = false;** would catch the unclosed **(** in the stack.
((A+B)+[C-D])	Y	 	In case 3, we have the same number of characters and are paired properly. Therefore, it would output "The expression is balanced." Having the bracket would not cause it to break or cause a negative output because it is properly paired and the number of characters is also equal.
((A+B]+[C-D]}	N	 	In case 4, it is entirely different from case 2. As we see, we have the same number of characters, but upon

			closer inspection, some of the characters do not match. Therefore, the **isMatchingPair** function that was made would catch and immediately break the loop.

Conclusion

The activity made us learn about stacks and their unique behavior of being a LIFO (Last in, First out) and new keywords like push, pop, isFull, isEmpty, etc. We were also shown how these are used in real code, like arrays and linked lists.

The array-based stack was fairly straightforward, mainly because the push and pop could be somewhat understood as just stackArr[++top] and [top--], although we did have to think about the capacity for maxCap ahead of time. Meanwhile, the linked-list version was different in that it did not need a fixed capacity at all, but made it more dynamic by using a manual cleanup like delete and pointer handling, which the version of the array never actually had to deal with.

For the supplementary activity, I made an array-based version of the solution first, since from my observation during the procedure, the logic was quite the same for the linked list. Therefore, I just adapted some of the functions and changed a few, like using char instead of int. Testing it was something different by itself; the four expressions were unbalanced, and with the requirement of analyzing it, I had to look back on the code and read through lines like isMatchingPair to be able to come up with an answer. The whole code itself, like case 4, where the open and closed symbols were equal but it did not match the recent opening, made it immediately break.

I think I was fairly able to do well for the whole activity, albeit reusing some of the given code, like the stackList.h for the solution that I was able to come up with. My main area of improvement would likely be being more attentive to small details, like missing a return false and copying the isMatchingPair function itself, which caused some errors that I was not familiar with. Overall, I would like to get better at being able to see these kinds of issues myself, and understanding why an error is actually occurring rather than fixing it and moving on.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction, Explanation of Concepts, Formatting**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The extent of my use of AI was asking Grammarly AI to fix the formatting of sentences after I write my whole idea into a notepad. I also followed it up with grammar correction, as usual, to fix any grammatical errors. Lastly, using Claude AI was to explain some concepts and find errors while explicitly asking it not to give any hints or answers directly, even after failing to understand the actual concept or error itself.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.